

41039 Programming 1

Revision of Week 3: while, for, do-while, break and continue



Dr. Xuan-Bach Le

le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

How the next fifty minutes work

Same deal as before. Not a test. Nothing is recorded against you. Answering earns extra credit, and being wrong out loud earns it too.

Part 1 — on your own

- Six short questions, about a minute each
- Commit to an answer **before** the reveal: on paper, or out loud

Part 2 — in teams of 3 or 4

- Four rounds: **draw the flow, spot the errors, predict the output, write the code**
- One sheet of paper, one spokesperson. Every team that reports back earns credit
- The last one goes to the keyboard: one Java team, one Python team

These are hard on purpose. How many times? Does it ever stop? Are these two really the same? If you are tracing values on paper, you are doing it right.

For every loop on every slide: what makes it stop?

Outline

- 1 On your own
- 2 Team round 1: draw the flow
- 3 Team round 2: spot the errors
- 4 Team round 3: predict the output
- 5 Team round 4: write the code
- 6 Wrapping up

Predict the output

Alone ■ 60 seconds



```
// (a)
for (int i = 0; i < 10; i += 3)
    System.out.print(i + " ");
// (b)
int i = 10;
while (i > 0) {
    i -= 3;
    System.out.print(i + " ");
}
// (c)
int k = 5;
do {
    System.out.print(k + " ");
    k++;
} while (k < 5);
```

Predict the output

Alone ■ 60 seconds

```
// (a)
for (int i = 0; i < 10; i += 3)
    System.out.print(i + " ");
// (b)
int i = 10;
while (i > 0) {
    i -= 3;
    System.out.print(i + " ");
}
// (c)
int k = 5;
do {
    System.out.print(k + " ");
    k++;
} while (k < 5);
```

Answers

- (a) 0 3 6 9 — 12 fails the test
- (b) 7 4 1 -2 — the body changes *i* *before* printing it. The last value printed is one the condition never approved
- (c) 5 — false from the very start, but a do-while runs its body **at least once**

The condition is checked at one point only. Know where.

Predict the output: two that surprise

Alone ■ 60 seconds



```
// (d)
for (int j = 0; j < 3; j++);
{
    System.out.print(j);
}
```

```
# (e)
for i in range(3):
    for j in range(i):
        print(i, j)
```

Predict the output: two that surprise

Alone ■ 60 seconds

```
// (d)
for (int j = 0; j < 3; j++);
{
    System.out.print(j);
}
```

```
# (e)
for i in range(3):
    for j in range(i):
        print(i, j)
```

Answers

(d) Does not compile: cannot find symbol `j`. The `;` is the loop's whole (empty) body. The block after it is *not* in the loop, and `j` stopped existing when the loop ended

(e) 1 0, then 2 0, then 2 1. When `i` is 0, `range(0)` is empty: the inner loop runs **zero** times

Indentation fools the reader. It never fools the compiler.

Count it

Alone ■ 90 seconds



```
// (a) How many times does the body run?  
for (int i = 1; i <= 100; i *= 3) { ... }
```

```
// (b) How many times does count++ run?  
for (int i = 0; i < 4; i++)  
    for (int j = i; j < 4; j++)  
        count++;
```

```
// (c) How many times round?  
int n = 1000;  
while (n > 1) { n /= 2; }
```

```
// (d), (e) sum starts at 0. What is it after?  
for (int i = 0; i < 10; sum += i++);  
for (int i = 0; i < 10; sum += ++i);
```



```
// (a) How many times does the body run?  
for (int i = 1; i <= 100; i *= 3) { ... }  
  
// (b) How many times does count++ run?  
for (int i = 0; i < 4; i++)  
    for (int j = i; j < 4; j++)  
        count++;  
  
// (c) How many times round?  
int n = 1000;  
while (n > 1) { n /= 2; }  
  
// (d), (e) sum starts at 0. What is it after?  
for (int i = 0; i < 10; sum += i++);  
for (int i = 0; i < 10; sum += ++i);
```

Answers

- (a) **5**: i is 1, 3, 9, 27, 81. Then 243
- (b) **10** = $4 + 3 + 2 + 1$. The inner loop **starts** at the outer variable, so it shrinks each time
- (c) **9**: 500, 250, 125, 62, 31, 15, 7, 3, 1. Integer division — Week 1, still biting
- (d) **45**: $i++$ adds i , *then* increments: $0 + 1 + \dots + 9$
- (e) **55**: $++i$ increments *first*: $1 + 2 + \dots + 10$. Both loops ran ten times

Counting passes is tracing, not guessing. Write the values down.

What makes this stop?

Alone ■ 90 seconds

Stops, or runs forever? If it stops: after how many passes?

```
// (a)
int i = 0;
while (i < 10); { i++; }

// (b)
for (int i = 1; i != 10; i += 2) { ... }

// (c)
int i = 1;
while (i != 0) { i = (i + 1) % 5; }
```

# (d)	# (e)
i = 10	for i in range(5):
while i > 0:	i = i - 1
i - 1	

What makes this stop?

Alone ■ 90 seconds

Stops, or runs forever? If it stops: after how many passes?

```
// (a)
int i = 0;
while (i < 10); { i++; }

// (b)
for (int i = 1; i != 10; i += 2) { ... }

// (c)
int i = 1;
while (i != 0) { i = (i + 1) % 5; }
```

```
# (d)
i = 10
while i > 0:
    i - 1

# (e)
for i in range(5):
    i = i - 1
```

Answers

- (a) **✗ Forever.** The ; is the body, so i++ is never reached
- (b) **✗ Forever.** i is always odd: it steps over 10. i < 10 would stop
- (c) **✓ Stops, 4 passes:** 2, 3, 4, 0
- (d) **✗ Forever.** i - 1 is computed and thrown away. It needs i -= 1
- (e) **✓ Stops, 5 passes.** range hands out the next value, whatever you did to i

A condition with != is only safe if the variable is certain to land on the value.

break, continue, and labels

Alone ■ 90 seconds

```
outer:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == 2) continue;
        if (i == j) break;
        System.out.print(i + "," + j + " ");
    }
}
```

What prints? One change at a time

- A. as written
- B. with break → break outer
- C. with continue → break

break, continue, and labels

Alone ■ 90 seconds

```
outer:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == 2) continue;
        if (i == j) break;
        System.out.print(i + "," + j + " ");
    }
}
```

What prints? One change at a time

- A. as written
- B. with break → break outer
- C. with continue → break

Answers

A. 2,1 2,3 3,1

i = 1: j = 1 hits i == j and ends the row at once. i = 2: 1 prints, 2 is skipped, 3 prints. i = 3: 1 prints, 2 is skipped, 3 breaks

B. Nothing at all. The very first pair, 1,1, leaves *both* loops

C. 2,1 3,1 — every row now ends at j = 2, and only the *inner* loop ends

break ends the loop it is in. A label is the only way to say which one.

Loop-else: the search

Alone ■ 90 seconds

```
n = int(input("n: "))
d = 2
while d * d <= n:
    if n % d == 0:
        print(n, "is divisible by", d)
        break
    d += 1
else:
    print(n, "is prime")
```

Questions

1. What prints for 91? 97? 2? 1?
2. Delete the break. What prints for 36?

Loop-else: the search

Alone ■ 90 seconds

```
n = int(input("n: "))
d = 2
while d * d <= n:
    if n % d == 0:
        print(n, "is divisible by", d)
        break
    d += 1
else:
    print(n, "is prime")
```

Questions

1. What prints for 91? 97? 2? 1?
2. Delete the break. What prints for 36?

Answers

1. 91: 91 is divisible by 7 — break skips the else
97: 97 is prime — the condition fails, the else runs
2: 2 is prime — **zero** passes, and the else *still* runs
1: 1 is prime — **X wrong**, as for 0 and negatives. Loop-else cannot tell “searched, found nothing” from “never searched”
2. Divisible by 2, 3, 4, 6 — *and then* 36 is prime. With no break, the else always runs

Read loop-else as “if the loop was never broken out of”.

Outline

- 1 On your own
- 2 Team round 1: draw the flow**
- 3 Team round 2: spot the errors
- 4 Team round 3: predict the output
- 5 Team round 4: write the code
- 6 Wrapping up

Form teams

Now

- Teams of **3 or 4**, with the people next to you
- One sheet of paper. Write the team's names on it
- Pick a **spokesperson** — a different one for each challenge, if you can

For each challenge

- The timer is on the slide
- When it runs out, one team is called on to answer. Others may disagree — **disagreeing with a reason also earns credit**
- “I think it's X *because...*” beats “X”

How to work as a team on a trace

One person reads the code aloud, one keeps the table of variable values, and everyone else checks every line of the table. Most wrong answers come from one skipped step, not from misunderstanding.

Draw it: the flowchart

Teams ■ 5 min



```
int n;  
do {  
    System.out.print("n: ");  
    n = sc.nextInt();  
} while (n < 2);  
int count = 0;  
for (int i = 2; i <= n; i++) {  
    boolean prime = true;  
    for (int j = 2; j * j <= i; j++) {  
        if (i % j == 0) {  
            prime = false;  
            break;  
        }  
    }  
    if (prime) count++;  
}  
System.out.println(count);
```

Use four shapes

Start / End

input, output

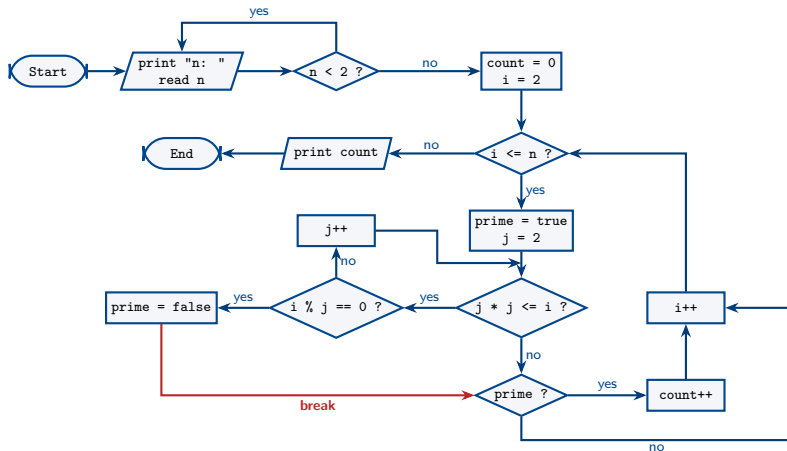
do something

question?

Draw it, then answer

1. Draw the whole flowchart. Every for unpacks into three shapes — which?
2. The do-while and the for: where is each one's diamond, compared with its body?
3. Where does the break arrow go?
4. How many arrows point *into* the diamond $j * j \leq i$? Into $i \leq n$?

Draw it: one answer



1. Start value, diamond, update — $i = 2$, $i \leq n$?, $i++$

2. do-while: diamond **after** the body. for: diamond **before** it

3. Out of the inner loop only — to prime ?, skipping $j++$

4. Two each: one from the start value, one from the update. Every loop has an arrow going **back**

Outline

- 1 On your own
- 2 Team round 1: draw the flow
- 3 Team round 2: spot the errors**
- 4 Team round 3: predict the output
- 5 Team round 4: write the code
- 6 Wrapping up



X Should print every prime below n, then how many.

```
1 int n = sc.nextInt();
2
3 next:
4 for (int i = 1; i <= n; i++) {
5     int count = 0;
6     for (int j = 2; j < i; i++) {
7         if (i % j == 0) break;
8     }
9     count++;
10    System.out.println(i + " is prime.");
11 }
12 System.out.println(count + " primes below " + n);
```

Three tasks

1. There are **five** bugs. Find them all, with line numbers.
2. Only one of the five stops it compiling. Which?
3. Fix *only* that one. Now what does it print for $n = 10$? (Trace it. It is not what you expect.)

For $n = 10$ it should print 2, 3, 5, 7 and 4 primes below 10.

Spot the errors 1: answers

The five bugs

L4 `i = 1`: 1 is not prime — and the inner loop never runs for it

L4 `i <= n`: “below `n`” means `i < n`

L5 count inside the loop: reset to 0 every pass, and gone after the loop — line 12

does not compile

L6 `i++` in the inner header: it must change **its own** variable, `j`

L7 `break` ends only the inner loop, so lines 9–10 still run. It needs `continue next` — the unused label was a clue

3. With only the compile error fixed, `n = 10`:

```
1 is prime.  
2 is prime.  
4 is prime.  
6 is prime.  
8 is prime.  
10 is prime.  
6 primes below 10
```

At `i = 3` the inner loop bumps `i` to 4, then `4 % 2 == 0` breaks — and the *even* number is announced. The bugs work together to produce almost exactly the wrong list.

The compiler found one bug in five. The other four needed a trace.

Spot the errors 2: odd numbers, in Python Teams ■ 3 min

X Should sum the odd numbers from 1 to n.

```
1 n = input("n: ")
2 total = 0
3 i = 1
4 while (i <= n) {
5     if i % 2 == 0:
6         continue
7     total += i
8     i++
9 }
10 print("Sum of odd numbers up to " + n
11       + " is " + total)
```

Written by someone who learnt Java first. For $n = 10$ it should print Sum of odd numbers up to 10 is 25.

Two tasks

1. Find all **five** errors, with line numbers.
2. Python shows you **one error at a time**: fix it, run again, meet the next. In what **order** will you meet the five? And which one never shows an error message at all?

Spot the errors 2: answers, in the order you meet them

Run, fix, run again

- L4, L9** { } — `SyntaxError`. A Python block is a colon and indentation
- L8** `i++` — `SyntaxError`. Python has no `++`: `i += 1`
- L1** no `int()` — `TypeError` on L4: `'<='` not supported between `int` and `str`
- L6** `continue` — **X no message at all**. At `i = 2` it skips `i += 1`, every time. The program just hangs
- L10** `"..." + n` — `TypeError`: can only concatenate `str` (not `int`). Use commas. It was there all along, hidden behind number 4

✓ Fixed

```
n = int(input("n: "))
total = 0
i = 1
while i <= n:
    if i % 2 == 1:
        total += i
    i += 1
print("Sum of odd numbers up to", n,
      "is", total)
```

Add the odd ones instead of skipping the even ones: no `continue`, no trap.

Syntax errors come first, run-time errors next, and the silent one last.



X It compiles. It runs. Five logic errors.

```
1 int n = sc.nextInt();
2 int limit = sc.nextInt();
3 int total = 0;
4 int sum = 0;
5 for (int i = 1; i < n; i++) {
6     for (int j = 1; j < i; j++) {
7         sum += j;
8         if (sum > limit) break;
9     }
10    System.out.println(i + ": " + sum);
11    total = sum;
12 }
13 System.out.println("Total: " + total);
```

Should: for each row i from 1 to n , print $1 + 2 + \dots + i$. Stop **everything** the moment a sum goes over limit. Then print the total of the rows printed.

5, 100 $\rightarrow$ 1: 1 ... 5: 15, Total: 35

5, 8 $\rightarrow$ 1: 1, 2: 3, 3: 6, Total: 10

Three tasks

1. Find all five (line numbers). Fix each.
2. What does it print for 5, 100?
3. For 5, 8 it ends Total: 10 — correct! Why is that no comfort?

Spot the errors 3: answers

The five, and their fixes

L4 `sum` is never reset: declare `int sum = 0;`
inside the outer loop, so each row starts from 0

L5 `i < n` misses the last row: `i <= n`

L6 `j < i` misses `i` itself: `j <= i`

L8 `break` ends only the *inner* loop: the row is still printed and the next row starts. Label the outer loop `rows:` and `break rows;`

L11 `total = sum` keeps only the last row:
`total += sum`

2. The buggy version, for 5, 100

```
1: 0
2: 1
3: 4
4: 10
Total: 10
```

3. For 5, 8 it prints `Total: 10` too — by accident, after four wrong rows. One right number proves nothing: **check every line**, and test more than one input.

Every one of these compiles. Only a trace, or a test, finds them.

Outline

- 1 On your own
- 2 Team round 1: draw the flow
- 3 Team round 2: spot the errors
- 4 Team round 3: predict the output**
- 5 Team round 4: write the code
- 6 Wrapping up

Predict 1: four loops, two claims

Teams ■ 4 min

Pair A

```
for (int i = 0; i < 10; i++) {  
    if (i % 3 == 0) continue;  
    System.out.print(i + " ");  
}
```

```
int i = 0;  
while (i < 10) {  
    if (i % 3 == 0) continue;  
    System.out.print(i + " ");  
    i++;  
}
```

Pair B (n is read from the user)

```
int k = 0;  
while (k < n) {  
    System.out.print(k + " ");  
    k++;  
}
```

```
int k = 0;  
do {  
    System.out.print(k + " ");  
    k++;  
} while (k < n);
```

For each pair

Write exactly what each of the four loops prints — pair B for $n = 3$, then $n = 0$. Which pair behaves the same for **every** input?

Predict 1: answers

X Pair A: not the same

- The for prints 1 2 4 5 7 8
- The while prints **nothing, forever**. At $i = 0$, continue jumps straight back to the condition — **past** the $i++$ at the bottom
- In a for, continue jumps to the **modification part**, so $i++$ still happens

✓ One fix

```
int i = 0;
while (i < 10) {
    if (i % 3 == 0) { i++; continue; }
    System.out.print(i + " ");
    i++;
}
```

X Pair B: same until $n \leq 0$

- For $n = 3$, both print 0 1 2
- For $n = 0$ (or any negative), the while prints nothing and the do-while prints 0
- The difference only shows when the condition is false **the first time**

The lesson

“Equivalent” means for every input. One counter-example is enough to break the claim — and it is almost always a boundary: zero passes, or a continue.

Predict 2: the mystery loop

Teams ■ 4 min



```
int n = sc.nextInt();
int r = 0;
while (n > 0) {
    r = r * 10 + n % 10;
    n = n / 10;
}
System.out.println(r);
```

Trace it for 472 with a table:

pass	n % 10	r	n
start	—	0	472
1			
...			

Four tasks

1. Finish the trace. What prints?
2. In one sentence: what does this loop compute?
3. What prints for 1200? 7? 0? -45? Which of those are wrong answers?
4. Someone adds, after the loop:
if (r == n)
System.out.println("palindrome");
What prints for 1221? Why?

Predict 2: answers

pass	n % 10	r	n
start	—	0	472
1	2	2	47
2	7	27	4
3	4	274	0

1. 274 2. It **reverses the digits**: % 10 takes the last digit off, / 10 throws it away, $r * 10$ makes room.

3. $1200 \rightarrow 21$: r is a number, and a number has no leading zeros. $7 \rightarrow 7$. $0 \rightarrow 0$. $-45 \rightarrow 0$ — **X wrong**: $n > 0$ is false at once, zero passes.

4. Only 1221 — **no** palindrome. After the loop n is **0**, so it compares 1221 with 0. (For input 0 it *does* say palindrome — right, by accident.) Save a copy first:

```
int n = sc.nextInt();
int original = n;           // missing
int r = 0;
while (n > 0) {
    r = r * 10 + n % 10;
    n = n / 10;
}
if (r == original)
    System.out.println("palindrome");
```

A loop that changes its variable uses it up. Keep a copy if you need it after.

Outline

- 1 On your own
- 2 Team round 1: draw the flow
- 3 Team round 2: spot the errors
- 4 Team round 3: predict the output
- 5 Team round 4: write the code**
- 6 Wrapping up



Java

```
outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (i + j == 3) break outer;
        System.out.println(i + " " + j);
    }
}
```

Two tasks

1. Write down the Java's full output.
2. Write Python that prints **exactly** the same. Python has no labelled break — and a plain break is *not* the same thing. Check your version against your answer to 1.

Hard bonus. Do task 3 *without* a new variable. (Hint: a loop can have an else.)

Code it 1: answers

1. Java: five lines

0 0, 0 1, 0 2, 1 0, 1 1.

At 1 2 it leaves **both** loops.

The obvious translation — just break — prints **six**: the same five, **plus** 2 0. It ends only the inner loop, so the outer one carries on with $i = 2$.

✓ 2. With a flag

```
found = False
for i in range(3):
    for j in range(3):
        if i + j == 3:
            found = True
            break
    print(i, j)
if found:
    break
```

✓ Bonus: loop-else

```
for i in range(3):
    for j in range(3):
        if i + j == 3:
            break
    print(i, j)
else:
    continue
break
```

Inner loop finished normally: the else runs, continue to the next i. Inner loop broken out of: the else is skipped and the outer break runs.

Clever — but most people will need a minute to read it. The flag is clearer.

Code it 2: Floyd's triangle, with a limit

Teams ■ 6 min

Read `rows` and `limit`. Print Floyd's triangle — row 1 has one number, row 2 has two, and the numbers keep counting up. **Stop everything** the moment the next number would be over `limit`.

- **One team** codes it in **Java** (`Floyd.java`), **one team** in **Python** (`floyd.py`), on the projector
- Every other team writes *both* on paper, and shouts fixes
- Python has no labelled break. Remember *Code it 1*

Run it with all of these

input	should print
5, 12	1 / 2 3 / 4 5 6 / 7 8 9 10 / 11 12
4, 100	four full rows
5, 10	four rows, then nothing
8, 12	as 5, 12 — no blank lines
3, 0	nothing at all

Code it 2: one answer in each language

✓ Java

```
int k = 1;
done:
for (int r = 1; r <= rows; r++) {
    for (int c = 1; c <= r; c++) {
        if (k > limit) break done;
        System.out.print(k + " ");
        k++;
    }
    System.out.println();
}
```

✓ Python

```
k = 1
done = False
for r in range(rows):
    for c in range(r + 1):
        if k > limit:
            done = True
            break
        print(k, end=" ")
        k += 1
    if done:
        break
    print()
```

Same idea, two toolkits: a label in Java, a flag in Python.

Code it 2: three ways to get it wrong

X Plain break

Leaves only the current row.
For rows 8, limit 12, rows 6, 7 and 8 each start, break at once, and print **three blank lines**. The 5, 12 test cannot see this bug.

X $k = 1$ inside the outer loop

```
1
1 2
1 2 3
1 2 3 4
```

Every row restarts. A counter that must *survive* the rows lives outside both loops.

X range(r) in Python

r starts at 0, so row 1 gets range(0) — **no numbers**. The first line is blank and every row is one short. Java counted r from 1; Python's range counts from 0.

The first test alone misses the plain break. That is why there were five.

Outline

- 1 On your own
- 2 Team round 1: draw the flow
- 3 Team round 2: spot the errors
- 4 Team round 3: predict the output
- 5 Team round 4: write the code
- 6 Wrapping up

You should now be able to...

Week 3

- ✓ say, for any loop, **what makes it stop** — or that nothing does
- ✓ count the passes of a loop, including a nested one whose inner bound uses the outer variable
- ✓ tell a `while` from a `do-while` on the input that runs **zero** times
- ✓ unpack a `for` into a `while` — and spot the `continue` that breaks the unpacking
- ✓ predict which loop a `break` or `continue` affects, with and without a label
- ✓ leave nested loops in Python with a **flag**
- ✓ explain when a `loop-else` runs, and when it lies
- ✓ trace a loop in a table, and keep a copy of what it uses up

If any box is unticked

Sections 05–07, quiz questions 9–15 and exercises 4–5:

lexuanbach.github.io/41039